

第十三章：构建工具链

一、核心概念

1.1 交叉编译（Cross Compilation）

在一种架构（Host）上编译出另一种架构（Target）的可执行文件。

```
Host: x86-64 mac0S/Linux
Target: riscv64
工具链: riscv64-unknown-elf-gcc
```

为什么需要交叉编译？ 目标平台（RISC-V 开发板/QEMU）通常没有足够的资源运行编译器。

1.2 编译的四个阶段

```
源码 (.c)
→ 预处理 (-E): 展开宏、头文件包含 → .i
→ 编译 (-S): C → 汇编 → .s
→ 汇编 (-c): 汇编 → 目标文件 → .o
→ 链接 (ld): 目标文件 + 库 → 可执行文件 (ELF)
```

1.3 链接脚本（Linker Script）

控制可执行文件的内存布局：

- **ENTRY(_entry)**：指定入口函数
- **. = 0x80200000**：指定加载地址
- **SECTIONS**：定义各段的排列顺序和属性
- **PROVIDE(symbol = .)**：定义链接器符号（供 C 代码引用）

1.4 Makefile 关键概念

概念	说明
目标 (target)	要生成的文件
依赖 (prerequisites)	生成目标需要的文件
命令 (recipe)	如何从依赖生成目标
隐式规则	%.o: %.c 自动推导
变量	CC, CFLAGS, LDFLAGS
并行编译	-j\$(nproc) 多核编译

1.5 裸机工具链 vs Linux 工具链

工具链	目标	特点
riscv64-unknown-elf-gcc	裸机	无 libc，用于内核/bootloader
riscv64-linux-musl-gcc	Linux + musl	有 musl libc，用于用户态程序
riscv64-linux-gnu-gcc	Linux + glibc	有 glibc，标准 Linux 用户态

1.6 objcopy 与二进制格式

objcopy -O binary 将 ELF 转为纯二进制，去掉所有 ELF 头和段信息。用于嵌入到内核中 (initcode.bin) 或烧录到 ROM。

二、基本推论

推论 1：链接脚本的加载地址必须与固件约定一致。OpenSBI 跳转到 0x80200000，所以内核链接脚本必须 = 0x80200000。地址不匹配会导致内核无法启动。

推论 2：-ffreestanding -nostdlib 是内核编译的必需选项。内核不能依赖标准库（没有 libc），-ffreestanding 告诉编译器不要假设有标准库函数，-nostdlib 不链接标准库。

推论 3：-mcmodel=medany 允许内核在高地址运行。RISC-V 的 medlow 代码模型只能寻址 ±2GB，medany 允许代码在任意 2GB 窗口内（适合内核地址 0x80200000+）。

三、Linux / 通用实现

3.1 Linux 内核构建系统 (Kbuild)

Linux 使用 Kbuild 系统：

- Kconfig：配置系统 (make menuconfig)，生成 .config
- Makefile 层级：每个目录有自己的 Makefile，递归构建
- obj-y / obj-m：编译到内核 / 编译为模块
- ccflags-y：目录级编译选项

```
# drivers/net/Makefile
obj-$(CONFIG_VIRTIO_NET) += virtio_net.o
```

3.2 GCC 常用内核编译选项

选项	作用
-Wall -Werror	所有警告 + 警告变错误
-O2	优化级别（内核通常用 O2）
-fno-stack-protector	关闭栈保护（内核自己管理）

选项	作用
<code>-fno-omit-frame-pointer</code>	保留帧指针（方便调试）
<code>-mno-relax</code>	禁止链接器松弛优化

3.3 QEMU 启动参数

参数	含义
<code>-machine virt</code>	QEMU virt 平台（标准 RISC-V 虚拟机）
<code>-bios default</code>	使用内置 OpenSBI
<code>-kernel xxx</code>	指定内核镜像
<code>-m 128M</code>	内存大小
<code>-smp 2</code>	CPU 核数
<code>-nographic</code>	无图形，串口输出
<code>-drive file=xxx</code>	磁盘镜像
<code>-netdev user,id=net0</code>	用户态网络

四、RUOS 的实现

4.1 Makefile 结构

```
TOOLPREFIX = riscv64-unknown-elf-
CFLAGS = -Wall -Werror -O -mcmodel=medany
        -ffreestanding -fno-common -nostdlib
        -march=rv64gc -mabi=lp64
MAKEFLAGS += -j$(NPROC) # 并行编译
```

4.2 编译流程

```
用户态: initcode.c + entry.S + ulib.o → initcode (ELF) → initcode.bin
内核:   src/**/*.c + src/**/*.S → build/**/*.o → kernel/kernel (ELF)
        kernel/kernel → kernel-qemu (评测要求名)
        kernel/kernel.asm (反汇编), kernel/kernel.sym (符号表)
```

4.3 第三方库处理

```
# lwext4: 禁用所有警告
LWEXT4_CFLAGS := -w -Wno-int-conversion
```

```
# open-npstack: 警告但不报错
OPEN_NPSTACK_CFLAGS := -Wno-error -Wno-incompatible-pointer-types
```

4.4 调试流程

```
# 终端 1: QEMU 等待 GDB
make qemu-gdb

# 终端 2: GDB 连接
gdb-multiarch -x debug_riscv.gdb
# target remote 127.0.0.1:1234
# symbol-file kernel/kernel
```

五、八股 / 面试高频问题

Q: 编译的四个阶段? A: 预处理（宏展开、头文件包含）→ 编译（C→汇编）→ 汇编（汇编→目标文件 .o）→ 链接（.o + 库 → 可执行文件）。

Q: 静态链接和动态链接的区别? A: 静态链接在编译时将所有库代码合并到可执行文件中。动态链接在运行时由 ld.so 加载共享库并解析符号。静态文件大但无依赖，动态文件小但需要 .so 文件。

Q: ELF 文件格式的基本结构? A: ELF Header → Program Header Table（执行视图：LOAD/INTERP/DYNAMIC）→ Section Header Table（链接视图：.text/.data/.bss/.got/.plt）。内核加载 ELF 时只看 Program Header。

Q: 什么是位置无关代码（PIC）? A: 代码中不包含绝对地址，所有地址引用通过 GOT 间接访问。共享库必须是 PIC 的，才能被加载到任意地址。

Q: Makefile 中 = 和 := 的区别? A: = 是递归展开（延迟求值），:= 是简单展开（立即求值）。:= 更可预测，性能更好。

六、项目贡献亮点

- 完整的 Makefile 构建系统（并行编译、多目标、条件编译、外部库特殊处理）
- 理解链接脚本如何决定内核内存布局（kernel.ld: 0x80200000, bss_start/bss_end, end 符号）
- 适配 musl 交叉编译器，支持加载动态链接的测试程序

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
构建系统	单一 Makefile	Kbuild 层级系统	简单但不可扩展
优化级别	-O	-O2	调试友好
外部库	源码嵌入 + 特殊 CFLAGS	模块/子系统编译	简单粗暴但有效

决策	RUOS 选择	Linux 做法	权衡
用户态工具链	bare-metal + musl	glibc	教学环境够用